

UART Transmitter

Design Implementation and Verification Report

Contents

1	Introduction	2
2	Block Interface	2
3	Specification Summary	2
4	RTL Synthesis Schematics	3
4.1	Top-Level UART_TX Schematic	3
5	Verification Methodology	4
5.1	Testbench	4
5.2	Waveform Verification	5
5.3	Simulation Log Results	7

1 Introduction

A Universal Asynchronous Receiver/Transmitter (UART) is a block of circuitry responsible for implementing full-duplex serial communication.

Transmitting UART (UART_TX) converts parallel data presented by a master device (e.g. a CPU) into a serial bitstream and shifts it out on a single output line, following the standard UART framing convention:

START -- DATA[7:0] -- (optional PARITY) -- STOP

This report documents the RTL implementation of the UART_TX block, and the self-checking testbench used to validate the design against the specification.

2 Block Interface

The top-level UART_TX module exposes the following ports, as defined by the specification:

Port	Width	Description
CLK	1	UART TX clock signal
RST	1	Asynchronous active-low reset
PAR_TYP	1	Parity type (0 = even, 1 = odd)
PAR_EN	1	Parity enable
P_DATA	8	Input data byte
DATA_VALID	1	Input data valid strobe (high for 1 cycle)
TX_OUT	1	Serial data output
Busy	1	High during transmission, low otherwise

Table 1: UART_TX port list

3 Specification Summary

- New data is accepted on P_DATA only when DATA_VALID is high.
- All registers are cleared using an asynchronous active-low reset.
- DATA_VALID is asserted for exactly one clock cycle.
- Busy remains high for the entire duration of the frame transmission and low otherwise.
- The transmitter must not accept new data on P_DATA while a frame is in progress, even if DATA_VALID pulses again during that time.
- TX_OUT idles high when no transmission is in progress.
- PAR_EN: 0 disables the parity bit, 1 enables it.
- PAR_TYP: 0 selects even parity, 1 selects odd parity.

Each frame therefore consists of: one start bit (1'b0), 8 data bits (LSB first), an optional parity bit, and one stop bit (1'b1).

4 RTL Synthesis Schematics

This section presents the synthesized RTL schematics of the design, and that each sub-block synthesizes to the intended logic.

4.1 Top-Level UART_TX Schematic

Figure 1 shows the synthesized top-level schematic. `u_serializer` and `u_parity_calc` both receive `P_DATA` and `DATA_VALID` directly, while `u_Tx_FSM` consumes their outputs (`serializer_out`, `parity_calc_out`, `ser_done`) along with the configuration and control signals (`PAR_EN`, `PAR_TYP`, `CLK`, `RST`) to drive `Tx_out`, `ser_en`, and `Busy`. This confirms the structural partitioning.

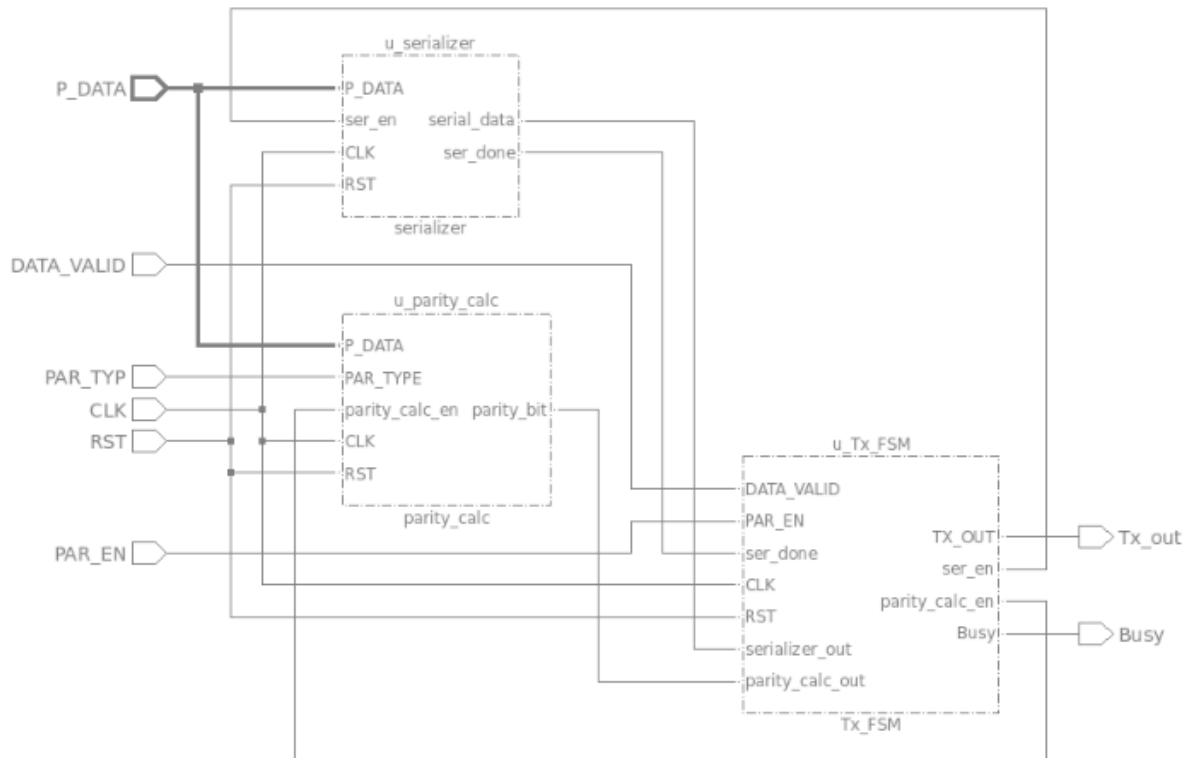


Figure 1: Synthesized RTL schematic of the top-level UART_TX module

- **Tx_FSM** – the controlling finite state machine (`IDLE` → `FRAME_START` → `DATA BITS` → `PARITY` (optional) → `FRAME_END` → `IDLE`), which also drives the output mux select and the `Busy` flag.
- **serializer** – shifts `P_DATA` out one bit at a time, LSB first, and asserts `ser_done` once all 8 bits have been shifted.
- **parity_calc** – computes the even/odd parity bit for the latched byte.

5 Verification Methodology

5.1 Testbench

A self-checking testbench (`UART_Tx_tb`) drives the `UART_TX` DUT at a 200 MHz clock (`CLK_CYCLE` = 5 ns, matching the requirement), generated by a free-running `always` block toggling every half period.

The testbench is built around three reusable tasks:

- **Reset_UART** – clears all DUT inputs (`P_DATA`, `DATA_VALID`, `PAR_EN`, `PAR_TYP`) to 0, asserts `RST_tb` = 0 (active low), and releases it (`RST_tb` = 1) on the following negative clock edge.
- **check_bit** – samples `Tx_out_tb` on the current negative clock edge and compares it against an expected value passed in as an argument, printing a `PASS` or `FAIL` line for that bit.
- **Send_and_check_Frame** – parameterized by `data`, `par_en`, `par_typ`, and a `corrupt_mid_frame` flag. It first computes the expected parity bit locally then drives `P_DATA`/`PAR_EN`/`PAR_TYP` and pulses `DATA_VALID` high for exactly one clock cycle.

It then calls `check_bit` once for the start bit, once per data bit (LSB first, 8 iterations), once for the parity bit if `par_en` is set, and once for the stop bit and finally waiting on `negedge Busy_tb` to confirm the DUT has returned to idle before the task returns.

If `corrupt_mid_frame` is set, the task forces `P_DATA` to the bitwise complement of the original byte immediately after checking data bit index 3, to verify that a write attempt mid-frame has no effect on the bits still being shifted out.

The main stimulus block resets the DUT and then exercises the following scenarios:

1. No-parity frame, 8'h55 (`PAR_EN` = 0), via `Send_and_check_Frame`.
2. Even-parity frame, 8'hAA (`PAR_TYP` = 0), via `Send_and_check_Frame`.
3. Odd-parity frame, 8'hF0 (`PAR_TYP` = 1), via `Send_and_check_Frame`.
4. Even-parity frame, 8'hA5, sent with `corrupt_mid_frame` = 1: `P_DATA` is overwritten with `~data` (8'h5A) partway through the frame while `Busy` = 1, verifying the already-latched byte continues to shift out unaffected.
5. A final scenario coded directly in the stimulus block (not via the task) to test spurious `DATA_VALID` handling: an even-parity frame of 8'h55 (`PAR_EN` = 1, `PAR_TYP` = 0) is sent, and once the start bit and the first three data bits have been checked, `DATA_VALID` is pulsed high again mid-frame together with a different byte (8'h53) and different parity settings (`PAR_EN` = 1, `PAR_TYP` = 1) while `Busy` = 1. The remaining data bits, the parity bit, and the stop bit are then checked against the values expected for the *original* 8'h55/even-parity frame, confirming the spurious request is ignored and the in-flight frame completes undisturbed.

5.2 Waveform Verification

Figures 2–6 show ModelSim waveform captures of P_DATA_tb, DATA_VALID_tb, ser_en, ser_done, Busy_tb, Tx_out_tb, parity_calc_en, and the FSM current_state/next_state signals across consecutive frames.

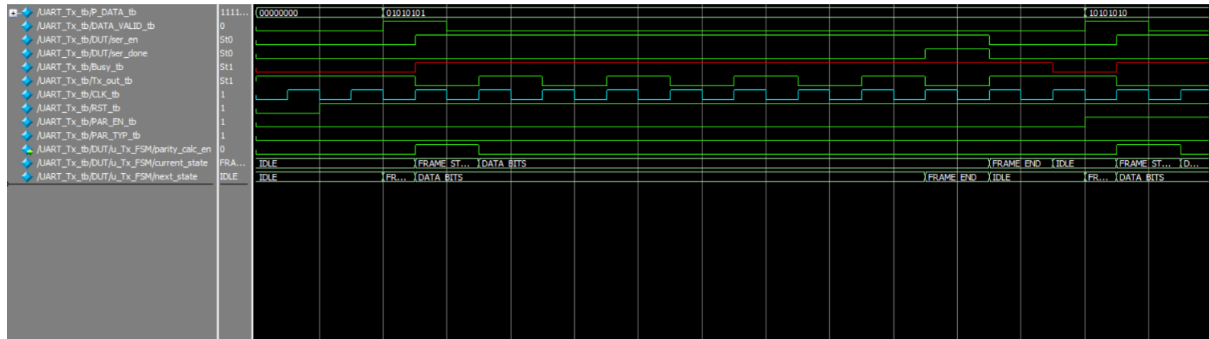


Figure 2: Frame 1: no-parity frame

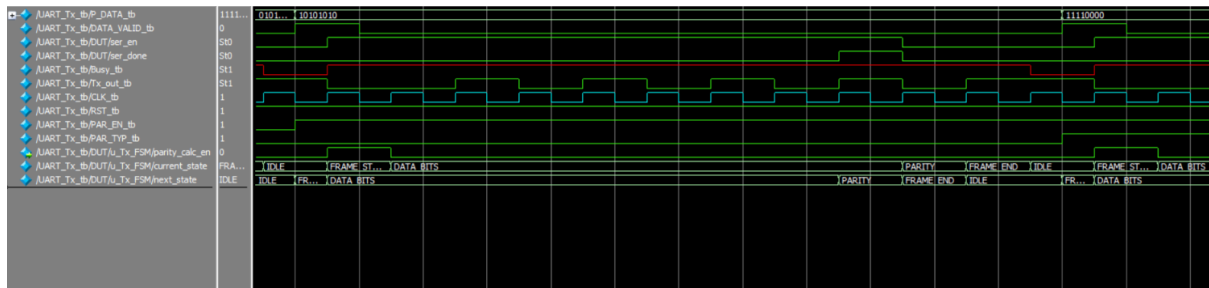


Figure 3: Frame 2: even-parity frame entering the PARITY state

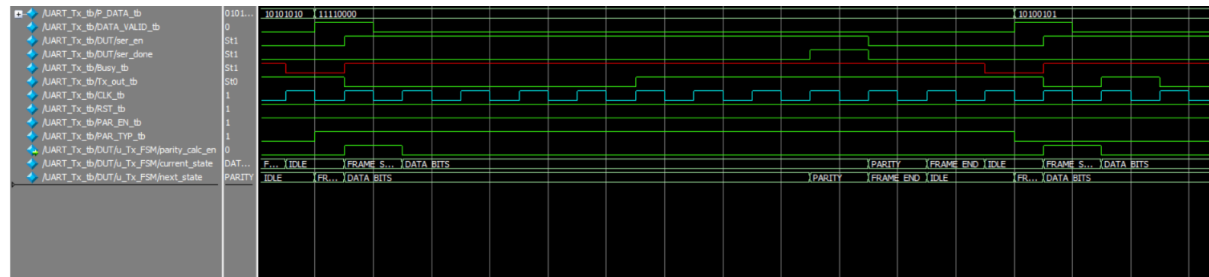


Figure 4: Frame 3: odd-parity frame

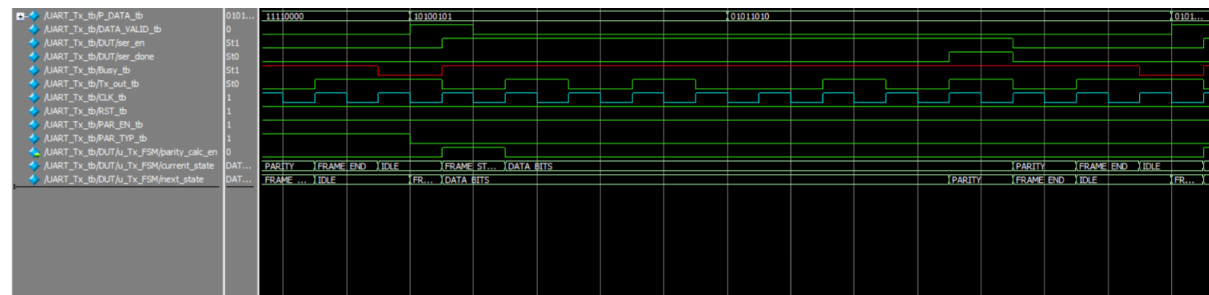


Figure 5: Frame 4: mid-frame P_DATA corruption test, output unaffected

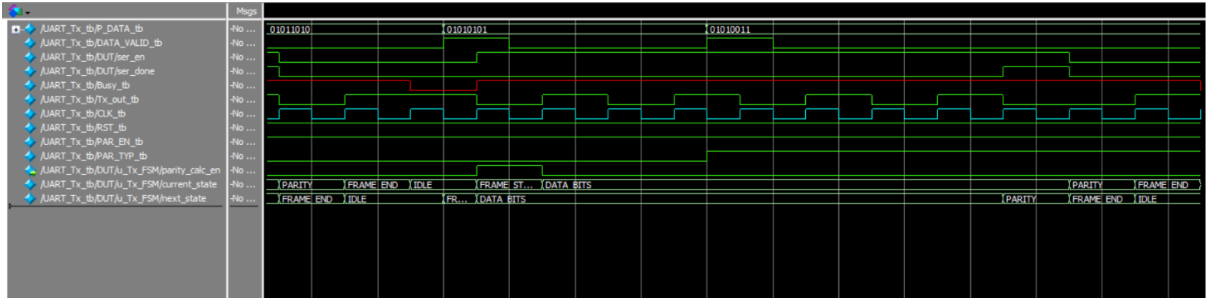


Figure 6: Frame 5: spurious DATA_VALID pulse mid-frame, frame completes undisturbed

In every capture, the FSM correctly sequences through IDLE → FRAME_START → DATA_BITS → PARITY (when enabled) → FRAME_END → IDLE, with Busy_tb held high for the full frame duration and Tx_out_tb shifting the data bits out LSB-first between the start and stop bits.

5.3 Simulation Log Results

The console transcript below captures the self-checking testbench output for all five frames described in Section 5.1. Each frame reports the simulation timestamp, the byte being sent, the parity configuration, and a PASS/FAIL verdict for every bit compared by `check_bit`.

```
=====
[10000] Sending Frame
  DATA = 55 (01010101)
  PAR_EN = 0
  PAR_TYP = 0
-----
  PASS: expected 0, got 0
  PASS: expected 1, got 1
  PASS: expected 0, got 0
  PASS: expected 1, got 1
  PASS: expected 0, got 0
  PASS: expected 1, got 1
  PASS: expected 0, got 0
  PASS: expected 0, got 0
  PASS: expected 1, got 1
=====
[65000] Sending Frame
  DATA = aa (10101010)
  PAR_EN = 1
  PAR_TYP = 0
-----
  PASS: expected 0, got 0
  PASS: expected 0, got 0
  PASS: expected 1, got 1
  PASS: expected 0, got 0
  PASS: expected 1, got 1
  PASS: expected 0, got 0
  PASS: expected 1, got 1
  PASS: expected 0, got 0
  PASS: expected 1, got 1
=====
[125000] Sending Frame
  DATA = f0 (11110000)
  PAR_EN = 1
  PAR_TYP = 1
-----
  PASS: expected 0, got 0
  PASS: expected 0, got 0
  PASS: expected 0, got 0
  PASS: expected 0, got 0
  PASS: expected 0, got 0
  PASS: expected 1, got 1
  PASS: expected 1, got 1
  PASS: expected 1, got 1
  PASS: expected 1, got 1
  PASS: expected 1, got 1
=====
[185000] Sending Frame
  DATA = a5 (10100101)
  PAR_EN = 1
  PAR_TYP = 0
-----
```

```

PASS: expected 0, got 0
PASS: expected 1, got 1
PASS: expected 0, got 0
PASS: expected 1, got 1
PASS: expected 0, got 0
[mid-frame] P_DATA driven to 5a while Busy=1
PASS: expected 0, got 0
PASS: expected 1, got 1
PASS: expected 0, got 0
PASS: expected 1, got 1
PASS: expected 0, got 0
PASS: expected 1, got 1
=====
[243000] Sending Frame (spurious DATA_VALID mid-frame test)
DATA = 55 (01010101)
PAR_EN = 1
PAR_TYP = 0
-----
PASS: expected 0, got 0
PASS: expected 1, got 1
PASS: expected 0, got 0
PASS: expected 1, got 1
[mid-frame] DATA_VALID pulsed with P_DATA=51 while Busy=1 (must be ignored)
PASS: expected 0, got 0
PASS: expected 1, got 1
PASS: expected 0, got 0
PASS: expected 1, got 1
PASS: expected 0, got 0
PASS: expected 1, got 1
=====
UART TEST COMPLETED
=====

```